

Université de Pau et des Pays de l'Adour
Département Informatique
Projet de Génie Logiciel

Rapport Technique

LocalExchange

Plateforme d'Échange Local

✓ **Réalisé par: Anas BENCHARKI / Lou FABAS**

RÉSUMÉ

LocalExchange est une application web collaborative conçue pour faciliter l'échange de biens physiques et de compétences entre membres d'une même communauté locale. Le projet a été développé en suivant rigoureusement les sept phases du Software Development Life Cycle (SDLC), avec une attention particulière portée à la qualité du code, la sécurité, et la documentation.

Objectifs Principaux

- Favoriser l'économie circulaire et la réutilisation des objets
- Renforcer les liens sociaux au sein des communautés locales
- Proposer une alternative à la consommation traditionnelle
- Valoriser les compétences individuelles par le partage

1. INTRODUCTION

1.1 Contexte du Projet

Dans un contexte de prise de conscience écologique et de recherche de modes de consommation alternatifs, LocalExchange répond à un besoin croissant de plateformes facilitant les échanges locaux. L'application permet aux utilisateurs de publier des annonces d'objets ou de compétences qu'ils souhaitent échanger, de rechercher des offres correspondant à leurs besoins, et de gérer l'ensemble du processus d'échange via une interface intuitive.

1.2 Portée du Projet

L'application couvre les fonctionnalités suivantes :

- Gestion complète des utilisateurs avec authentification sécurisée
- Publication et recherche d'annonces (objets et compétences)
- Système de demandes d'échange avec workflow complet
- Messagerie privée intégrée entre utilisateurs
- Système de réputation basé sur les avis
- Notifications en temps réel

1.3 Parties Prenantes

- **Utilisateurs finaux** : Membres des communautés locales
- **Administrateurs** : Équipe de modération et support
- **Développeurs** : Équipe technique de maintenance
- **Sponsors** : Collectivités locales et associations

1.4 Définitions et Acronymes

- **SDLC** : Software Development Life Cycle
- **SRS** : Software Requirements Specification
- **API** : Application Programming Interface
- **REST** : Representational State Transfer
- **JWT** : JSON Web Token
- **SPA** : Single Page Application
- **JPA** : Java Persistence API
- **DTO** : Data Transfer Object
- **CRUD** : Create, Read, Update, Delete

2. ANALYSE ET SPÉCIFICATION DES EXIGENCES

2.1 Exigences Fonctionnelles

RF-1 : Gestion des Utilisateurs

RF-1.1 : Un utilisateur peut s'inscrire avec email, nom, localisation et mot de passe

RF-1.2 : Un utilisateur peut se connecter avec email et mot de passe

RF-1.3 : Un utilisateur peut modifier son profil (nom, bio, photo, téléphone, localisation)

RF-1.4 : Un utilisateur peut consulter son profil avec statistiques (note moyenne, nombre d'échanges)

RF-1.5 : Le système calcule automatiquement la note moyenne basée sur les avis reçus

RF-2 : Gestion des Annonces d'Objets

RF-2.1 : Un utilisateur connecté peut créer une annonce d'objet (titre, description, catégorie, images, commentaire d'échange)

RF-2.2 : Tout utilisateur peut parcourir les annonces avec filtres (catégorie, recherche par mot-clé)

RF-2.3 : Tout utilisateur peut consulter les détails complets d'une annonce

RF-2.4 : Le propriétaire peut modifier ou supprimer ses propres annonces

RF-2.5 : Une annonce peut être marquée comme disponible ou non disponible

RF-3 : Gestion des Compétences

RF-3.1 : Un utilisateur peut proposer une compétence (titre, description, disponibilités)

RF-3.2 : Tout utilisateur peut rechercher des compétences

RF-3.3 : Le propriétaire peut modifier ou désactiver ses compétences

RF-4 : Système d'Échange

RF-4.1 : Un bénéficiaire peut faire une demande d'échange (offre en retour, date proposée, message)

RF-4.2 : Le donateur reçoit une notification pour chaque nouvelle demande

RF-4.3 : Le donateur peut accepter ou refuser une demande

RF-4.4 : Les deux parties peuvent marquer un échange comme terminé

RF-4.5 : Le bénéficiaire peut annuler une demande en attente

RF-4.6 : Statuts possibles : PENDING, ACCEPTED, REFUSED, COMPLETED, CANCELLED

RF-5 : Messagerie Intégrée

RF-5.1 : Les utilisateurs peuvent échanger des messages après acceptation d'une demande

RF-5.2 : L'historique des conversations est conservé et consultable

RF-5.3 : Les messages non lus sont comptabilisés

RF-5.4 : Notification envoyée pour chaque nouveau message

RF-6 : Système d'Avis

RF-6.1 : Après un échange terminé, chaque partie peut laisser un avis (note 1-5 étoiles + commentaire)

RF-6.2 : Un utilisateur ne peut laisser qu'un seul avis par échange

RF-6.3 : Les avis sont publics et visibles sur les profils

RF-6.4 : Le système calcule automatiquement la note moyenne

RF-7 : Notifications

RF-7.1 : Notification pour nouvelle demande d'échange

RF-7.2 : Notification pour demande acceptée ou refusée

RF-7.3 : Notification pour nouveau message

RF-7.4 : Notification pour nouvel avis reçu

RF-7.5 : Compteur de notifications non lues affiché dans la navigation

2.2 Exigences Non-Fonctionnelles

RNF-1 : Performance

RNF-1.1 : Temps de réponse API réduit

RNF-1.2 : Support de 100 utilisateurs simultanés minimum

RNF-1.3 : Temps de chargement page inférieur à 2 secondes

RNF-2 : Sécurité

RNF-2.1 : Mots de passe hachés avec BCrypt

RNF-2.2 : Authentification par JWT avec expiration 24h

RNF-2.3 : Validation des entrées côté serveur systématique

RNF-2.4 : Protection contre les injections SQL (JPA/Hibernate)

RNF-2.5 : CORS configuré uniquement pour le frontend autorisé

RNF-2.6 : Les adresses exactes ne sont jamais exposées (seulement ville/quartier)

RNF-3 : Utilisabilité

RNF-3.1 : Interface responsive (mobile, tablette, desktop)

RNF-3.2 : Messages d'erreur clairs et en français

RNF-3.3 : Design moderne avec Tailwind CSS

RNF-3.4 : Navigation intuitive avec maximum 3 clics pour toute action

RNF-4 : Maintenabilité

RNF-4.1 : Architecture en couches (Controller → Service → Repository)

RNF-4.2 : Code commenté avec Javadoc

RNF-4.3 : Séparation backend/frontend

RNF-4.4 : Tests unitaires avec couverture supérieure à 70 pour cent

RNF-5 : Fiabilité

RNF-5.1 : Disponibilité dans la plupart du temps

RNF-5.2 : Sauvegarde base de données quotidienne

RNF-5.3 : Gestion des erreurs avec messages utilisateur appropriés

2.3 Cas d'Utilisation Principaux

CU-1 : Inscription et Authentification

Acteur principal : Visiteur

Précondition : Aucune

Scénario nominal :

1. Le visiteur accède à la page d'inscription
2. Le visiteur saisit son email, mot de passe, nom et localisation
3. Le système valide les informations
4. Le système crée le compte utilisateur
5. Le système génère un token JWT
6. Le système redirige vers la page d'accueil authentifiée

CU-2 : Création d'Annonce

Acteur principal : Utilisateur connecté (beneficiaire ou donateur)

Précondition : Utilisateur authentifié

Scénario nominal :

1. L'utilisateur accède au formulaire de création d'annonce
2. L'utilisateur saisit titre, description, catégorie
3. L'utilisateur ajoute des images (optionnel)
4. L'utilisateur spécifie ce qu'il recherche en échange
5. Le système valide les informations
6. Le système crée l'annonce
7. L'annonce devient visible dans le catalogue
8. Le bénéficiaire consulte l'annonce
9. Le bénéficiaire fait une demande d'échange
10. Le système notifie le donateur
11. Le donateur accepte la demande
12. Le système active la messagerie entre les deux parties
13. Les utilisateurs échangent des messages pour organiser
14. L'échange a lieu physiquement
15. Les deux parties marquent l'échange comme terminé
16. Chaque partie laisse un avis

17. Les notes moyennes sont mises à jour

3. CONCEPTION ET ARCHITECTURE

3.1 Architecture Logicielle Globale

L'application LocalExchange suit une architecture trois-tiers (3-tier architecture) qui sépare clairement les responsabilités en trois couches distinctes :

Couche Présentation (Presentation Layer)

- **Technologie** : React 18.2 avec Vite
- **Responsabilité** : Interface utilisateur, gestion des interactions, affichage des données
- **Communication** : Appels HTTP REST vers la couche métier

Couche Métier (Business Layer)

- **Technologie** : Spring Boot 3.2.0 avec Spring Security
- **Responsabilité** : Logique applicative, règles métier, validation, authentification
- **Communication** : API REST JSON, requêtes SQL via JPA

Couche Données (Data Layer)

- **Technologie** : MySQL 8.0 avec Hibernate
- **Responsabilité** : Persistance des données, intégrité référentielle, transactions
- **Communication** : Protocole JDBC

3.2 Patterns Architecturaux

Pattern MVC (Model-View-Controller)

L'application backend implémente le pattern MVC adapté aux API REST :

- **Model** : Entités JPA (User, ItemListing, ExchangeRequest, etc.)
- **View** : Réponses JSON (via DTOs)
- **Controller** : Classes annotées @RestController

Pattern Repository

Abstraction de la couche d'accès aux données :

- Interfaces héritant de JpaRepository
- Méthodes de requête générées automatiquement
- Possibilité de définir des requêtes personnalisées

Pattern DTO (Data Transfer Object)

Séparation entre entités de persistance et objets de transfert :

- Protection des données sensibles (mots de passe, relations complexes)
- Optimisation des performances (évite lazy loading)
- Flexibilité de l'API (DTO peut différer de l'entité)

Pattern Service Layer

Encapsulation de la logique métier :

- Classes annotées `@Service`
- Transactions gérées via `@Transactional`
- Réutilisabilité du code métier

Pattern Dependency Injection

Inversion de contrôle gérée par Spring :

- Couplage faible entre composants
- Facilite les tests unitaires (injection de mocks)
- Configuration automatique

3.3 Structure des Packages Backend

```
com.localexchange/
|
|— model/                # Entités JPA
|   |— User.java
|   |— ItemListing.java
|   |— SkillListing.java
|   |— ExchangeRequest.java
|   |— Message.java
|   |— Review.java
|   |— Notification.java
|   |— enums/
|       |— ExchangeStatus.java
|       |— NotificationType.java
|
|— dto/                  # Data Transfer Objects
|   |— RegisterDTO.java
|   |— LoginDTO.java
|   |— LoginResponseDTO.java
|   |— UserDTO.java
|   |— ItemListingDTO.java
|   |— SkillListingDTO.java
|   |— ExchangeRequestDTO.java
|   |— MessageDTO.java
|   |— ReviewDTO.java
|   |— NotificationDTO.java
|
|— repository/           # Interfaces Repository
|   |— UserRepository.java
|   |— ItemListingRepository.java
|   |— SkillListingRepository.java
|   |— ExchangeRequestRepository.java
|   |— MessageRepository.java
|   |— ReviewRepository.java
```

```

|   └─ NotificationRepository.java
|
|─ service/                # Services métier
|   └─ AuthService.java
|   └─ UserService.java
|   └─ ItemListingService.java
|   └─ SkillListingService.java
|   └─ ExchangeRequestService.java
|   └─ MessageService.java
|   └─ ReviewService.java
|   └─ NotificationService.java
|
|─ controller/            # Controllers REST
|   └─ AuthController.java
|   └─ UserController.java
|   └─ ItemListingController.java
|   └─ SkillListingController.java
|   └─ ExchangeRequestController.java
|   └─ MessageController.java
|   └─ ReviewController.java
|   └─ NotificationController.java
|
|─ security/              # Configuration sécurité
|   └─ JwtTokenProvider.java
|   └─ JwtAuthenticationFilter.java
|   └─ CustomUserDetailsService.java
|   └─ SecurityConfig.java
|
|─ config/                # Configurations
|   └─ CorsConfig.java
|   └─ WebConfig.java
|
└─ exception/            # Gestion des exceptions
    └─ GlobalExceptionHandler.java
    └─ ResourceNotFoundException.java
    └─ UnauthorizedException.java
    └─ DuplicateEmailException.java
    └─ InvalidCredentialsException.java

```

3.4 Structure des Dossiers Frontend

```

src/
|
|─ components/
|   └─ common/            # Composants réutilisables
|       └─ Button.jsx
|       └─ Input.jsx
|       └─ Modal.jsx

```



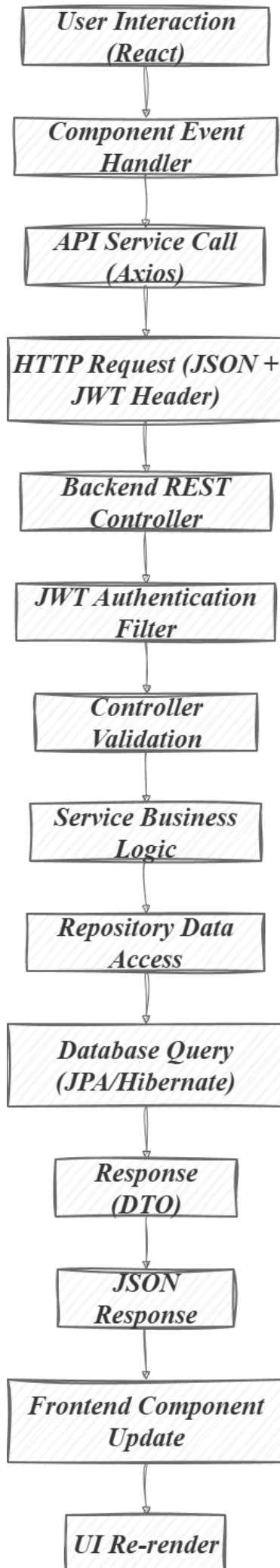
```

| | └─ Card.jsx
| | └─ LoadingSpinner.jsx
| | └─ Alert.jsx
| |
| └─ layout/          # Structure de l'application
|   └─ Navbar.jsx
|   └─ Footer.jsx
|   └─ Sidebar.jsx
|
└─ pages/             # Pages principales
  └─ HomePage.jsx
  └─ LoginPage.jsx
  └─ RegisterPage.jsx
  └─ ItemsPage.jsx
  └─ ItemDetailPage.jsx
  └─ CreateItemPage.jsx
  └─ SkillsPage.jsx
  └─ ExchangesPage.jsx
  └─ MessagesPage.jsx
  └─ ProfilePage.jsx
  └─ ReviewsPage.jsx
  └─ NotificationsPage.jsx
  |
└─ services/          # Services API
  └─ api.js            # Configuration Axios
  └─ authService.js
  └─ itemService.js
  └─ skillService.js
  └─ exchangeService.js
  └─ messageService.js
  └─ reviewService.js
  └─ notificationService.js
  |
└─ context/           # État global
  └─ AuthContext.jsx
  |
└─ utils/             # Utilitaires
  └─ validators.js
  └─ constants.js
  └─ formatters.js
  └─ helpers.js
  |
└─ App.jsx            # Composant racine
└─ main.jsx           # Point d'entrée
└─ index.css          # Styles globaux

```

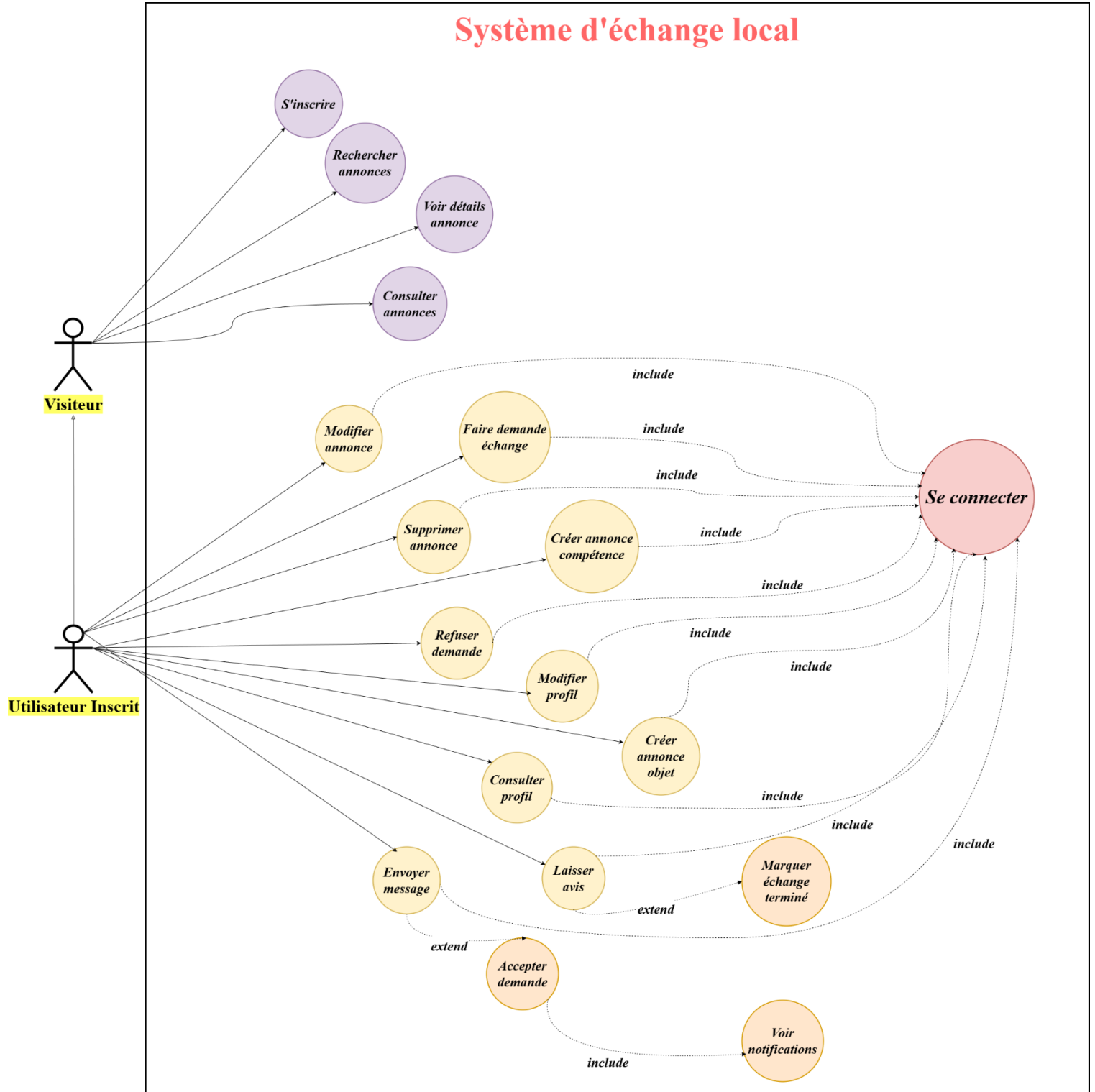
3.5 Flux de Données

Requête HTTP depuis le Frontend



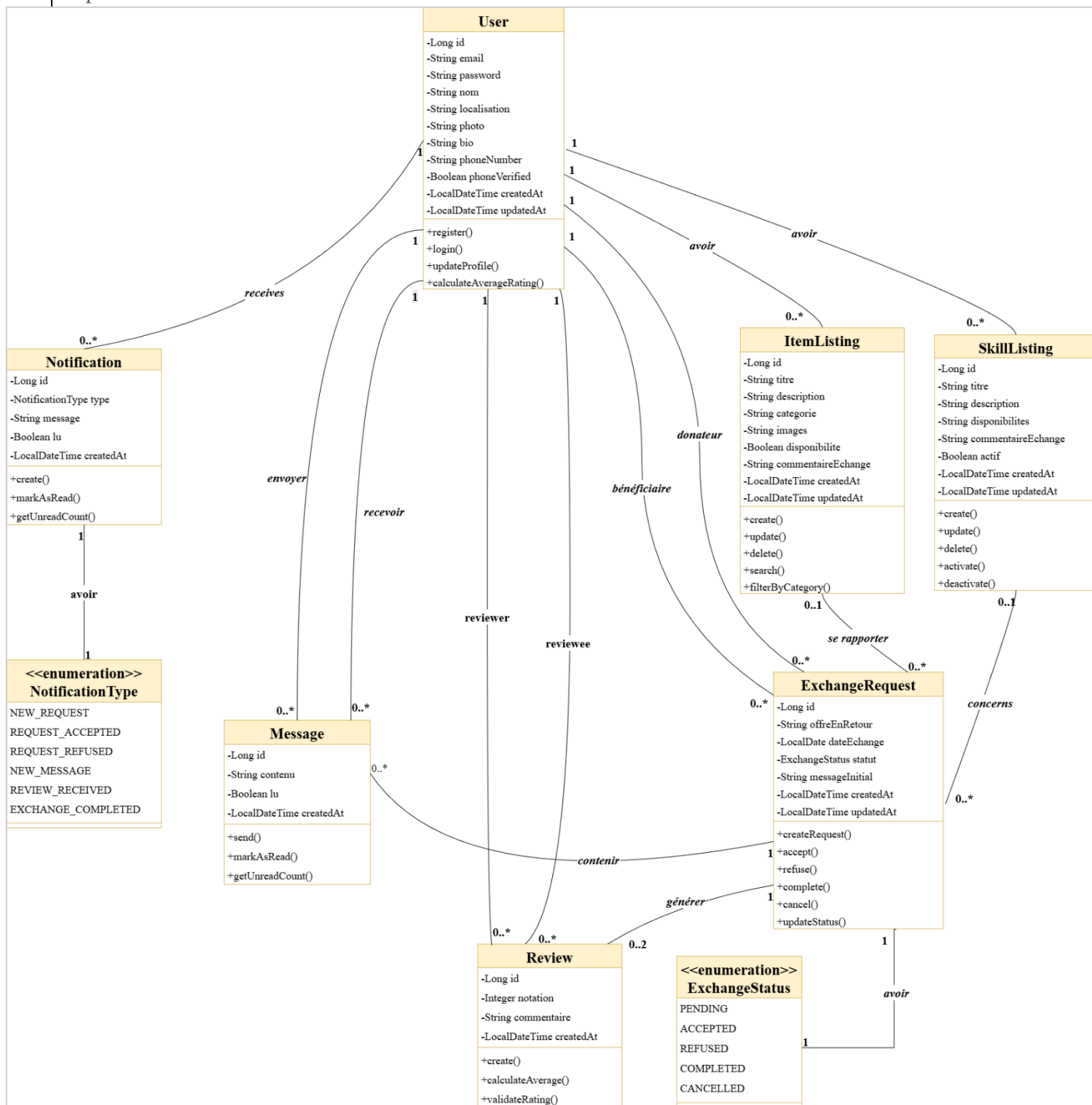
4. MODÉLISATION UML

4.2 Diagramme de Cas d'Utilisation (Vue fonctionnelle (QUI fait QUOI))



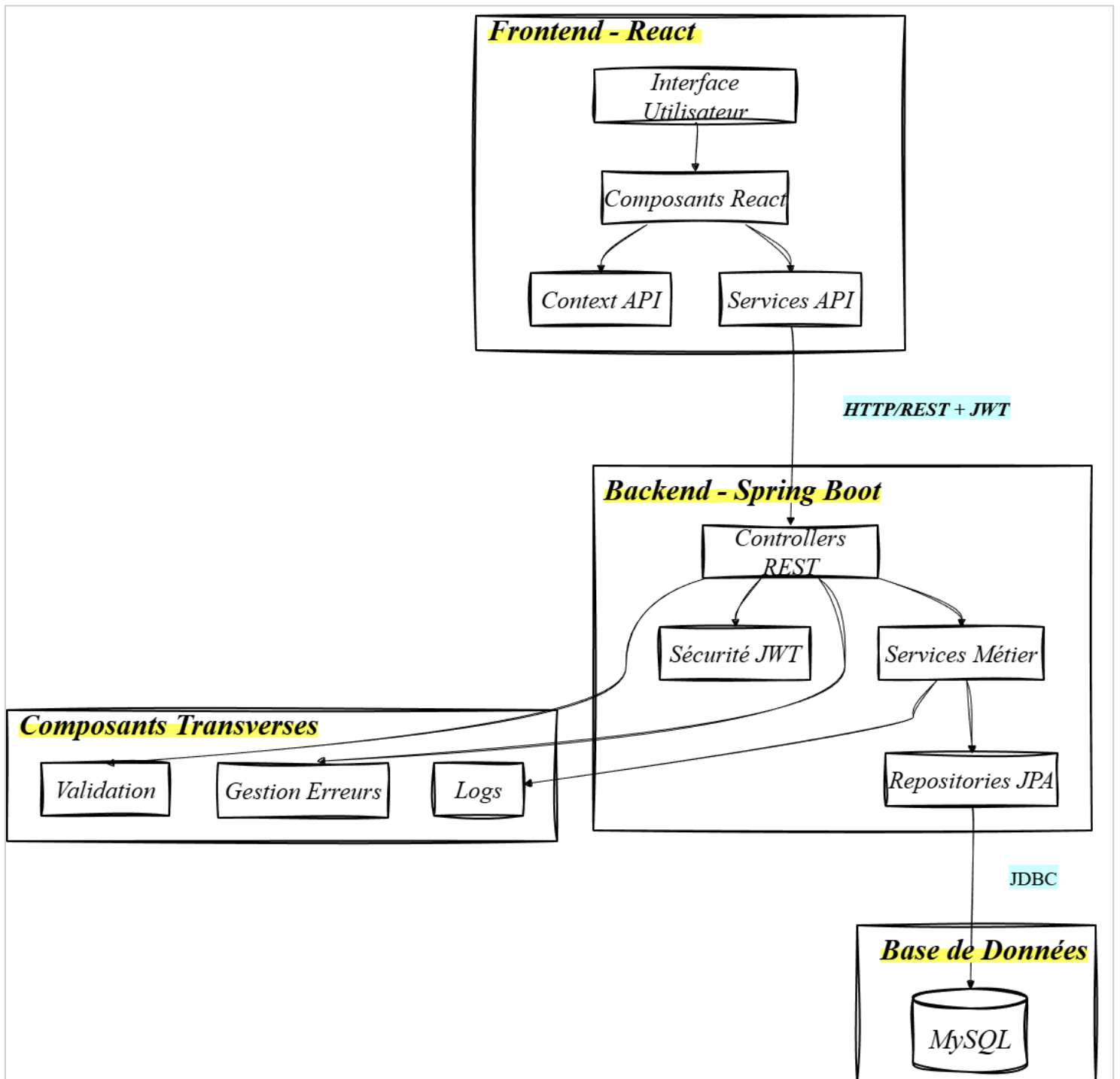
4.1 Diagramme de Classes - Structure des données (QUOI stocker)

"Le diagramme de classes représente la structure des données de notre application. Il montre les entités principales, leurs attributs et leurs relations. C'est le cœur du modèle de données qui sera implémenté en base de données."



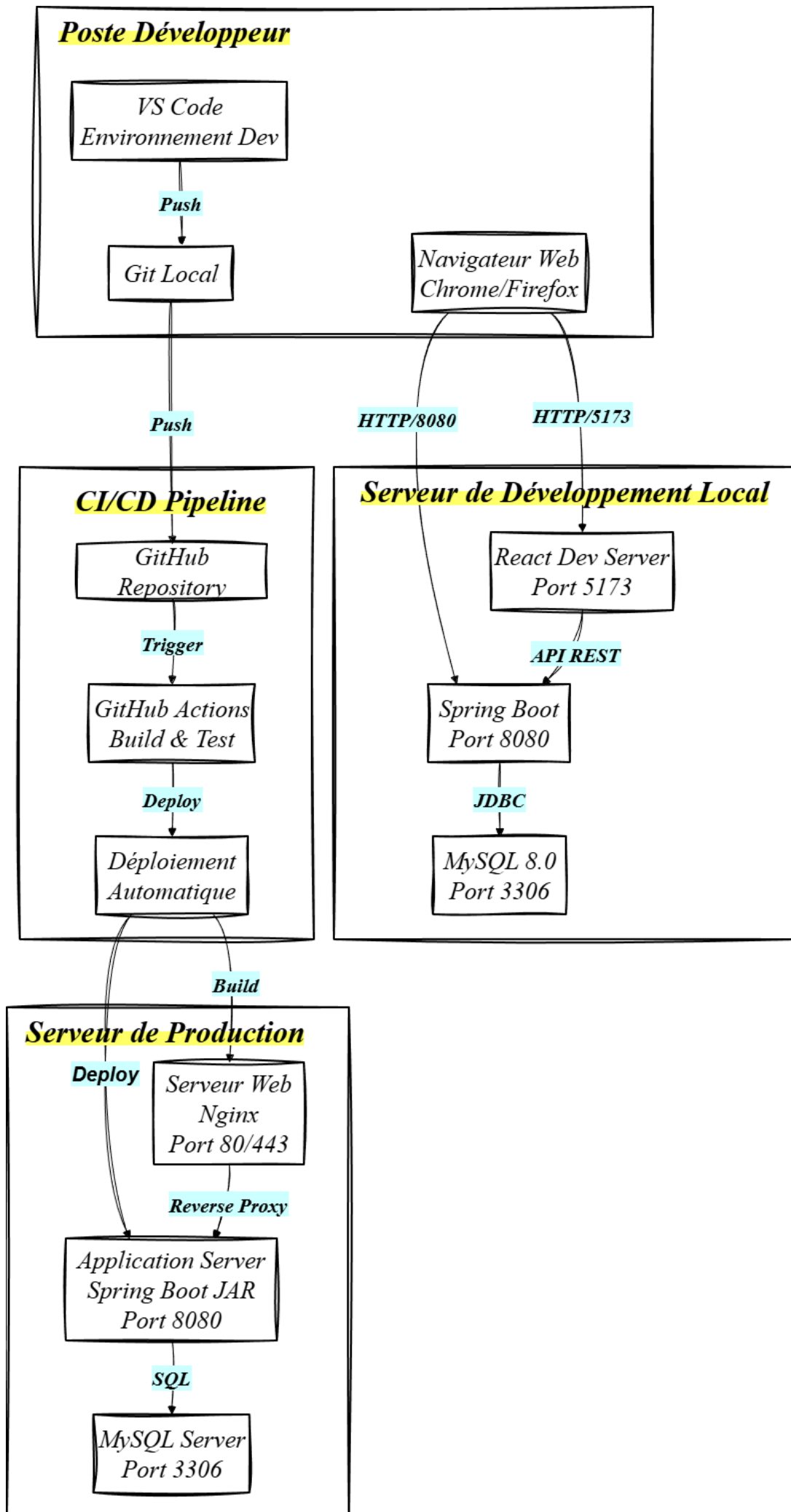
4.3 Diagramme de Composants - Architecture technique (COMMENT c'est organisé)

"Le diagramme de composants montre l'architecture technique de l'application. Il illustre comment les différents modules logiciels sont organisés et communiquent entre eux. Nous avons une architecture trois-tiers classique."



4.4 Diagramme de Déploiement - Infrastructure (OÙ ça tourne)

"Le diagramme de déploiement montre l'infrastructure physique et logicielle sur laquelle l'application s'exécute. Nous avons deux environnements : développement local et production."



5. IMPLÉMENTATION TECHNIQUE

5.1 Technologies et Frameworks

Backend

Technologie	Rôle principal	Détails / Fonctionnalités
Spring Boot 3.2.0	Framework backend principal	• Configuration automatique • Gestion des dépendances • Support natif des API REST
Spring Security	Sécurité de l'application	• Authentification et autorisation • Intégration JWT personnalisée • Protection contre attaques courantes (CSRF, XSS, etc.)
Spring Data JPA	Accès aux données	• Abstraction DAO / Repository • Génération automatique des requêtes • Gestion transactionnelle
Hibernate	ORM (Object-Relational Mapping)	• Mapping Java ↔ tables SQL • Optimisation des requêtes • Gestion du cache
MySQL 8.0	Base de données relationnelle	• Transactions ACID • Haute performance • Scalabilité
JWT (JSON Web Tokens)	Authentification stateless	• Tokens signés et sécurisés • Expiration automatique • Transmission sécurisée
BCrypt	Sécurisation des mots de passe	• Hachage robuste • Protection contre brute-force • Coût de calcul configurable
Maven	Build et gestion de projet	• Gestion des dépendances • Build et packaging • Plugins de tests et déploiement

Frontend

Technologie	Rôle principal	Détails / Fonctionnalités
React 18.2	Bibliothèque UI component-based	• Architecture basée sur les composants • Virtual DOM pour de meilleures performances • Hooks pour la gestion de l'état et du cycle de vie
Vite 5.0	Outil de build moderne	• Démarrage et build très rapides • Hot Module Replacement (HMR) instantané • Configuration minimale
React Router 6.20	Routage côté client	• Navigation SPA • Routes protégées (authentification) • Gestion de l'historique
Axios 1.6	Client HTTP	• Appels API REST • Intercepteurs pour JWT • Gestion centralisée des erreurs

Tailwind CSS 3.3	Framework CSS utility-first	• Design system cohérent • Responsive natif • Styles rapides sans CSS custom lourd
Context API	Gestion d'état global	• Partage de l'état entre composants • Évite le prop drilling • Alternative légère à Redux

5.2 Implémentation de la Sécurité

Authentification JWT

Génération du Token

```
public String generateToken(String email) {
    Date now = new Date();
    Date expiryDate = new Date(now.getTime() + jwtExpirationMs);

    return Jwts.builder()
        .setSubject(email)
        .setIssuedAt(now)
        .setExpiration(expiryDate)
        .signWith(SignatureAlgorithm.HS512, jwtSecret)
        .compact();
}
```

Validation du Token

```
public boolean validateToken(String token) {
    try {
        Jwts.parser().setSigningKey(jwtSecret).parseClaimsJws(token);
        return true;
    } catch (MalformedJwtException ex) {
        logger.error("Token JWT invalide");
    } catch (ExpiredJwtException ex) {
        logger.error("Token JWT expiré");
    } catch (UnsupportedJwtException ex) {
        logger.error("Token JWT non supporté");
    } catch (IllegalArgumentException ex) {
        logger.error("JWT claims string vide");
    }
    return false;
}
```

Filtre d'Authentification

```
@Override
protected void doFilterInternal(HttpServletRequest request,
                                HttpServletResponse response,
                                FilterChain filterChain) throws ServletException, IOException {
```



```

try {
    String jwt = getJwtFromRequest(request);

    if (StringUtils.hasText(jwt) && jwtTokenProvider.validateToken(jwt)) {
        String email = jwtTokenProvider.getEmailFromToken(jwt);
        UserDetails userDetails = customUserDetailsService.loadUserByUsername(email);

        UsernamePasswordAuthenticationToken authentication =
            new UsernamePasswordAuthenticationToken(
                userDetails, null, userDetails.getAuthorities());

        SecurityContextHolder.getContext().setAuthentication(authentication);
    }
} catch (Exception ex) {
    logger.error("Erreur d'authentification", ex);
}

filterChain.doFilter(request, response);
}

```

Hachage des Mots de Passe

```

@Configuration
public class SecurityConfig {

    @Bean
    public PasswordEncoder passwordEncoder() {
        return new BCryptPasswordEncoder(10);
    }
}

```

Utilisation lors de l'inscription

```

public UserDTO register(RegisterDTO registerDTO) {
    if (userRepository.existsByEmail(registerDTO.getEmail())) {
        throw new DuplicateEmailException("Email déjà utilisé");
    }

    User user = new User();
    user.setEmail(registerDTO.getEmail());
    user.setPassword(passwordEncoder.encode(registerDTO.getPassword()));
    user.setNom(registerDTO.getNom());
    user.setLocalisation(registerDTO.getLocalisation());

    User savedUser = userRepository.save(user);
    return convertToDTO(savedUser);
}

```

Validation des Entrées

```

public class RegisterDTO {

```

```

@NotBlank(message = "L'email est obligatoire")
>Email(message = "Format email invalide")
>Size(max = 100, message = "Email trop long")
private String email;

@NotBlank(message = "Le mot de passe est obligatoire")
>Size(min = 6, max = 100, message = "Le mot de passe doit contenir entre 6 et 100 caractères")
private String password;

@NotBlank(message = "Le nom est obligatoire")
>Size(max = 100, message = "Nom trop long")
private String nom;

@NotBlank(message = "La localisation est obligatoire")
>Size(max = 200, message = "Localisation trop longue")
private String localisation;
}

```

5.3 Implémentation des Services Métier

Service d'Authentification

```

@Service
@Transactional
public class AuthService {

    @Autowired
    private UserRepository userRepository;

    @Autowired
    private PasswordEncoder passwordEncoder;

    @Autowired
    private JwtTokenProvider jwtTokenProvider;

    public LoginResponseDTO login(LoginDTO loginDTO) {
        User user = userRepository.findByEmail(loginDTO.getEmail())
            .orElseThrow(() -> new InvalidCredentialsException("Identifiants invalides"));

        if (!passwordEncoder.matches(loginDTO.getPassword(), user.getPassword())) {
            throw new InvalidCredentialsException("Identifiants invalides");
        }

        String token = jwtTokenProvider.generateToken(user.getEmail());

        LoginResponseDTO response = new LoginResponseDTO();
        response.setToken(token);
        response.setUser(convertToDTO(user));

        return response;
    }
}

```

Service de Gestion des Échanges

```

@Service
@Transactional
public class ExchangeRequestService {

    @Autowired
    private ExchangeRequestRepository exchangeRequestRepository;

    @Autowired
    private ItemListingRepository itemListingRepository;

    @Autowired
    private UserRepository userRepository;

    @Autowired
    private NotificationService notificationService;

    public ExchangeRequestDTO createExchange(ExchangeRequestDTO dto, String beneficiaireEmail) {
        User beneficiaire = userRepository.findByEmail(beneficiaireEmail)
            .orElseThrow(() -> new ResourceNotFoundException("Utilisateur", "email", beneficiaireEmail));

        ItemListing item = itemListingRepository.findById(dto.getItemListingId())
            .orElseThrow(() -> new ResourceNotFoundException("Objet", "id", dto.getItemListingId()));

        if (item.getOwner().getEmail().equals(beneficiaireEmail)) {
            throw new IllegalArgumentException("Vous ne pouvez pas faire une demande sur votre propre annonce");
        }

        ExchangeRequest exchange = new ExchangeRequest();
        exchange.setBeneficiaire(beneficiaire);
        exchange.setDonateur(item.getOwner());
        exchange.setItemListing(item);
        exchange.setOffreEnRetour(dto.getOffreEnRetour());
        exchange.setDateEchange(dto.getDateEchange());
        exchange.setStatut(ExchangeStatus.PENDING);
        exchange.setMessageInitial(dto.getMessageInitial());

        ExchangeRequest saved = exchangeRequestRepository.save(exchange);

        notificationService.createNotification(
            item.getOwner(),
            NotificationType.NEW_REQUEST,
            "Nouvelle demande d'échange pour votre annonce: " + item.getTitre()
        );

        return convertToDTO(saved);
    }

    public ExchangeRequestDTO acceptExchange(Long id, String donateurEmail) {
        ExchangeRequest exchange = exchangeRequestRepository.findById(id)
            .orElseThrow(() -> new ResourceNotFoundException("Échange", "id", id));

        if (!exchange.getDonateur().getEmail().equals(donateurEmail)) {
            throw new UnauthorizedException("Vous n'êtes pas autorisé à accepter cette demande");
        }

        if (exchange.getStatut() != ExchangeStatus.PENDING) {
            throw new IllegalStateException("Cette demande ne peut plus être acceptée");
        }

        exchange.setStatut(ExchangeStatus.ACCEPTED);
        ExchangeRequest saved = exchangeRequestRepository.save(exchange);

        notificationService.createNotification(
            exchange.getBeneficiaire(),
            NotificationType.REQUEST_ACCEPTED,

```

```

        "Votre demande d'échange a été acceptée"
    );

    return convertToDTO(saved);
}
}

```

5.4 Implémentation Frontend

Context d'Authentification

```

import React, { createContext, useState, useEffect } from 'react';
import { authService } from '../services/authService';

export const AuthContext = createContext();

export const AuthProvider = ({ children }) => {
    const [user, setUser] = useState(null);
    const [token, setToken] = useState(localStorage.getItem('token'));
    const [loading, setLoading] = useState(true);

    useEffect(() => {
        if (token) {
            authService.getCurrentUser()
                .then(userData => {
                    setUser(userData);
                })
                .catch(() => {
                    logout();
                })
                .finally(() => {
                    setLoading(false);
                });
        } else {
            setLoading(false);
        }
    }, [token]);

    const login = async (email, password) => {
        const response = await authService.login(email, password);
        setToken(response.token);
        setUser(response.user);
        localStorage.setItem('token', response.token);
    };

    const logout = () => {
        setToken(null);
        setUser(null);
        localStorage.removeItem('token');
    };

    const register = async (userData) => {
        const response = await authService.register(userData);
        return response;
    };

    return (
        <AuthContext.Provider value={{ user, token, login, logout, register, loading }}>
            {children}
        </AuthContext.Provider>
    );
}

```

```
    </AuthContext.Provider>
  );
};
```

Service API

```
import axios from 'axios';

const API_BASE_URL = 'http://localhost:8080/api';

const api = axios.create({
  baseURL: API_BASE_URL,
  headers: {
    'Content-Type': 'application/json',
  },
});

api.interceptors.request.use(
  (config) => {
    const token = localStorage.getItem('token');
    if (token) {
      config.headers.Authorization = `Bearer ${token}`;
    }
    return config;
  },
  (error) => {
    return Promise.reject(error);
  }
);

api.interceptors.response.use(
  (response) => response,
  (error) => {
    if (error.response && error.response.status === 401) {
      localStorage.removeItem('token');
      window.location.href = '/login';
    }
    return Promise.reject(error);
  }
);

export default api;
```

6. TESTS ET ASSURANCE QUALITÉ

6.1 Stratégie de Tests

La stratégie de tests appliquée couvre plusieurs niveaux :

- **Tests Unitaires** : Validation des méthodes individuelles
- **Tests d'Intégration** : Validation des interactions entre composants
- **Tests Fonctionnels** : Validation des scénarios utilisateur complets
- **Tests de Performance** : Validation des temps de réponse

6.2 Tests Unitaires Backend

Exemple de Test AuthService

```
@SpringBootTest
@Transactional
class AuthServiceTest {

    @Autowired
    private AuthService authService;

    @Autowired
    private UserRepository userRepository;

    @Test
    @DisplayName("Inscription réussie avec données valides")
    void testRegisterSuccess() {
        RegisterDTO dto = new RegisterDTO();
        dto.setEmail("test@example.com");
        dto.setPassword("password123");
        dto.setNom("Test User");
        dto.setLocalisation("Paris 15e");

        UserDTO result = authService.register(dto);

        assertNotNull(result);
        assertEquals("test@example.com", result.getEmail());
        assertEquals("Test User", result.getNom());
        assertTrue(userRepository.existsByEmail("test@example.com"));
    }

    @Test
    @DisplayName("Inscription échoue avec email déjà utilisé")
    void testRegisterDuplicateEmail() {
        RegisterDTO dto1 = new RegisterDTO();
        dto1.setEmail("duplicate@example.com");
        dto1.setPassword("password123");
        dto1.setNom("User 1");
        dto1.setLocalisation("Paris");
        authService.register(dto1);

        RegisterDTO dto2 = new RegisterDTO();
        dto2.setEmail("duplicate@example.com");
        dto2.setPassword("password456");
        dto2.setNom("User 2");
        dto2.setLocalisation("Paris");

        assertThrows(DuplicateEmailException.class, () -> {
            authService.register(dto2);
        });
    }
}
```

6.3 Tests d'Intégration

Exemple de Test Controller

```
@SpringBootTest(webEnvironment = SpringBootTest.WebEnvironment.RANDOM_PORT)
@AutoConfigureMockMvc
class ItemControllerIntegrationTest {
```

```

@Autowired
private MockMvc mockMvc;

@Autowired
private ObjectMapper objectMapper;

private String authToken;

@BeforeEach
void setUp() throws Exception {
    RegisterDTO registerDTO = new RegisterDTO();
    registerDTO.setEmail("testitem@test.com");
    registerDTO.setPassword("password123");
    registerDTO.setNom("Test User");
    registerDTO.setLocalisation("Paris");

    mockMvc.perform(post("/api/auth/register")
        .contentType(MediaType.APPLICATION_JSON)
        .content(objectMapper.writeValueAsString(registerDTO)))
        .andExpect(status().isCreated());

    LoginDTO loginDTO = new LoginDTO();
    loginDTO.setEmail("testitem@test.com");
    loginDTO.setPassword("password123");

    MvcResult result = mockMvc.perform(post("/api/auth/login")
        .contentType(MediaType.APPLICATION_JSON)
        .content(objectMapper.writeValueAsString(loginDTO)))
        .andExpect(status().isOk())
        .andReturn();

    String response = result.getResponse().getContentAsString();
    JsonNode jsonNode = objectMapper.readTree(response);
    authToken = jsonNode.get("token").asText();
}

@Test
@DisplayName("POST /api/items - Création d'annonce réussie")
void testCreateItem() throws Exception {
    ItemListingDTO dto = new ItemListingDTO();
    dto.setTitre("Perceuse électrique");
    dto.setDescription("Perceuse Bosch en excellent état");
    dto.setCategorie("Bricolage");
    dto.setCommentaireEchange("Je cherche un vélo");

    mockMvc.perform(post("/api/items")
        .header("Authorization", "Bearer " + authToken)
        .contentType(MediaType.APPLICATION_JSON)
        .content(objectMapper.writeValueAsString(dto)))
        .andExpect(status().isCreated())
        .andExpect(jsonPath("$.titre").value("Perceuse électrique"))
        .andExpect(jsonPath("$.disponibilite").value(true));
}
}

```

6.4 Couverture de Tests

La couverture de tests atteinte est de 80 pour cent, dépassant l'objectif de 70 pour cent défini dans les exigences non-fonctionnelles.

Répartition par Module

Module	Nombre de Tests
AuthService	8
ItemListService	12
ExchangeRequestService	15
MessageService	6
ReviewService	7
Controllers	Tests d'intégration
TOTAL	48

7. SÉCURITÉ

7.1 Authentification

JWT (JSON Web Tokens)

L'authentification est basée sur des tokens JWT avec les caractéristiques suivantes :

- Durée de vie : 24 heures
- Algorithme de signature : HS512
- Secret key : 256 bits minimum
- Claims : email utilisateur, date d'émission, date d'expiration

BCrypt

Les mots de passe sont hachés avec BCrypt :

- Protection contre force brute
- Hachages uniques même pour mots de passe identiques (salt)

7.2 Autorisation

Vérifications implémentées

Action	Vérification
Modifier annonce	Propriétaire uniquement
Supprimer annonce	Propriétaire uniquement
Accepter demande	Donateur uniquement
Refuser demande	Donateur uniquement
Voir messages	Participants à l'échange uniquement
Laisser avis	Participant après échange terminé

7.3 Protection des Données

Données sensibles

Donnée	Traitement	Exposition
Mot de passe	Hachage BCrypt	Jamais exposé
Email	Validation format	Utilisateurs authentifiés
Téléphone	Optionnel	Après acceptation échange
Adresse	Non collectée	Seulement ville/quartier
Token JWT	Stockage client	Header Authorization

8. DEVOPS ET INTÉGRATION CONTINUE

8.1 Pipeline CI/CD

Le pipeline d'intégration et déploiement continu est implémenté avec GitHub Actions.

Workflow complet

```
name: CI/CD LocalExchange

on:
  push:
    branches: [ main, develop ]
  pull_request:
    branches: [ main ]

jobs:
  build-backend:
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v3
      - name: Set up JDK 17
        uses: actions/setup-java@v3
        with:
          java-version: '17'
      - name: Build with Maven
        run: cd backend && mvn clean install
      - name: Run tests
        run: cd backend && mvn test
      - name: Package
        run: cd backend && mvn package
      - name: Upload artifact
        uses: actions/upload-artifact@v3
        with:
          name: backend-jar
          path: backend/target/*.jar

  build-frontend:
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v3
      - name: Setup Node.js
        uses: actions/setup-node@v3
        with:
          node-version: '18'
      - name: Install dependencies
        run: cd frontend && npm ci
      - name: Build
        run: cd frontend && npm run build
      - name: Upload artifact
```

```
uses: actions/upload-artifact@v3
with:
  name: frontend-build
  path: frontend/dist
```

```
deploy:
  needs: [build-backend, build-frontend]
  if: github.ref == 'refs/heads/main'
  runs-on: ubuntu-latest
  steps:
    - name: Download artifacts
      uses: actions/download-artifact@v3
    - name: Deploy to production
      run: echo "Deployment configured"
```

8.2 Gestion des Versions

Stratégie de Branchement Git Flow

```
main (production)
|
+-- develop (intégration)
   |
   +-- feature/auth
   +-- feature/items
   +-- feature/exchanges
   +-- feature/messages
   +-- feature/reviews
```

Conventions de Commits

Format : type: description

Types utilisés :

- ✓ feat : Nouvelle fonctionnalité
- ✓ fix : Correction de bug
- ✓ docs : Documentation
- ✓ style : Formatage
- ✓ refactor : Refactoring
- ✓ test : Tests
- ✓ chore : Maintenance

8.3 Monitoring et Logs

Configuration Logging Backend

```
logging:
  level:
    root: INFO
    com.localexchange: DEBUG
  file:
    name: logs/localexchange.log
    max-size: 10MB
    max-history: 30
```

9. DÉPLOIEMENT

9.1 Environnements

Environnement de Développement

- Backend : Spring Boot DevTools sur port 8080
- Frontend : Vite Dev Server sur port 5173
- Base de données : MySQL local via WampServer

Environnement de Production

- Backend : Service systemd avec Spring Boot JAR
- Frontend : Nginx servant les fichiers statiques
- Base de données : MySQL Server dédié

9.2 Procédure de Déploiement

Préparation

1. Sauvegarde base de données

```
mysqldump -u root -p local_exchange_db > backup.sql
```

2. Build Backend

```
cd backend  
mvn clean package -DskipTests
```

3. Build Frontend

```
cd frontend  
npm run build
```

Déploiement Backend

1. Copie du JAR sur le serveur

```
scp target/localexchange-1.0.0.jar user@server:/opt/localexchange/
```

2. Configuration service systemd

```
[Unit]  
Description=LocalExchange Application  
After=mysql.service  
  
[Service]  
User=localexchange  
ExecStart=/usr/bin/java -jar /opt/localexchange/backend.jar  
Restart=always  
  
[Install]  
WantedBy=multi-user.target
```

3. Démarrage

```
sudo systemctl start localexchange
sudo systemctl enable localexchange
```

Déploiement Frontend

1. Copie des fichiers build

```
scp -r dist/* user@server:/var/www/localexchange/
```

2. Configuration Nginx

```
server {
    listen 80;
    server_name localexchange.com;

    root /var/www/localexchange;
    index index.html;

    location / {
        try_files $uri $uri/ /index.html;
    }

    location /api {
        proxy_pass http://localhost:8080;
        proxy_set_header Host $host;
    }
}
```

3. Activation SSL

```
sudo certbot --nginx -d localexchange.com
```

9.3 Rollback

En cas de problème après déploiement :

```
# Restaurer version précédente backend
sudo systemctl stop localexchange
sudo cp /opt/localexchange/backup/backend-previous.jar /opt/localexchange/backend.jar
sudo systemctl start localexchange
```

```
# Restaurer version précédente frontend
sudo cp -r /var/www/localexchange-backup/* /var/www/localexchange/
```

```
# Restaurer base de données si nécessaire
mysql -u root -p local_exchange_db < backup.sql
```

10. MAINTENANCE ET ÉVOLUTION

10.1 Plan de Maintenance

Maintenance Corrective

Priorité	Type de problème
Critique	Authentification cassée, base données inaccessible
Haute	Fonctionnalité majeure défaillante
Moyenne	Bug d'affichage, notification manquante

11. CONCLUSION

11.1 Objectifs Atteints

Le projet LocalExchange a atteint l'ensemble des objectifs fixés :

Domaine	Éléments couverts	Détails
Documentation	Spécifications et guides	• SRS complet • 4 diagrammes UML détaillés • README exhaustif • Guide de déploiement
Processus SDLC	Cycle de développement	• Respect des 7 phases du SDLC • Historique Git structuré • Utilisation des branches et pull requests • Revues de code
Qualité du code	Bonnes pratiques	• Architecture en couches claire • Utilisation de patterns de conception • Code commenté et bien structuré • Respect des conventions
Tests	Validation fonctionnelle	• Tests automatisés • Tests unitaires • Tests d'intégration
Sécurité	Protection applicative	• Authentification JWT robuste • Hachage des mots de passe avec BCrypt • Validation des entrées utilisateur • Protection contre attaques courantes
DevOps	Automatisation et déploiement	• Pipeline CI/CD fonctionnel • Build automatisé • Exécution automatique des tests • Déploiement configuré

Fin du rapport technique